CS-2001
Data Structures
Spring 2023

Linked List

Rizwan Ul Haq
Assistant Professor
FAST-NU
rizwan.haq@nu.edu.pk

Array Operations

- **Insertion**
  - Operation of adding another element to an array
  - How many steps in terms of n (number of elements in array)?
    - At the end
    - In the middle
    - In the beginning
  - O(n) steps at maximum (move items to insert at given location)
- **Deletion**
  - Operation of removing one of the elements from an array
  - How many steps in terms of n (number of elements in array)?
    - At the end
    - In the middle
    - In the beginning
  - O(n) steps at maximum (move items back to take place of deleted item)

Array Operations: Search Algorithms

- Linear or Sequential Search
  - Best case: constant time
  - Worst case: O(n)
  - Easy to implement and understand
  - Does not require any pre-sorted array
- Binary Search
  - Best case: constant time
  - Worst case: O(log(n))
  - Relatively difficult to implement
    - Requires sorted array

Limitation of Arrays

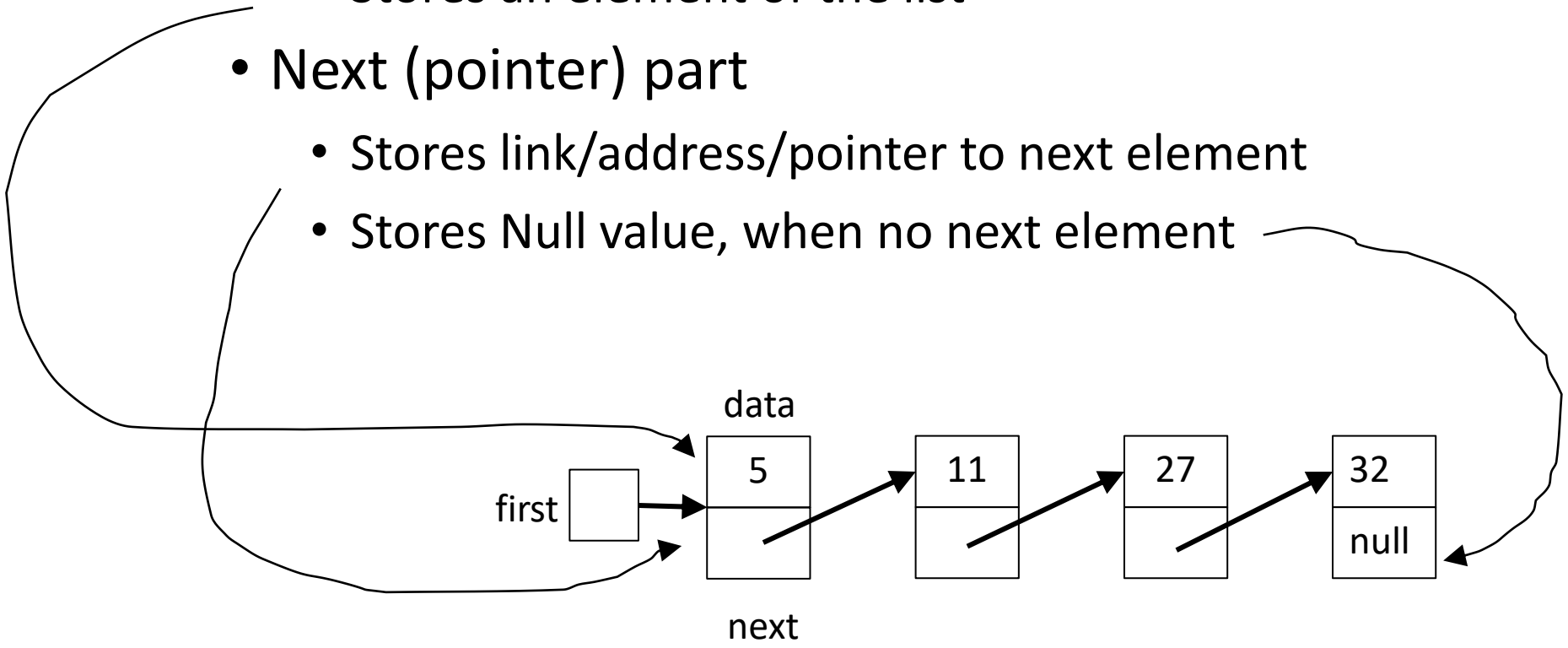
- An array has a limited number of elements
  - Routines inserting a new value have to check that there is room
- Can partially solve this problem by **reallocating** the array as needed (how much memory to add?)
  - Adding one element at a time could be costly
  - One approach - double the current size of the array
- A better approach: use a ***Linked List***

Pointers-Based Implementation of Lists

Linked List

Linked List

- Linked list nodes composed of two parts
  - Data part
    - Stores an element of the list
  - Next (pointer) part
    - Stores link/address/pointer to next element
    - Stores Null value, when no next element



Simple Linked List Class

- We use two classes: Node and List
- Declare Node class for the nodes
  - `data`: double-type data in this example
  - `next`: a pointer to the next node in the list

```
class Node {
public:
        double data;   // data
        Node* next;    // pointer to next Node
};
```

Simple Linked List Class

- Declare List, which contains
  - head: a pointer to the first node in the list
  - Since the list is empty initially, head is set to NULL

```
class List {
public:
        List(void) { head = NULL; } // constructor
        ~List(void);                // destructor
        bool IsEmpty() { return (head == NULL); }
        bool Insert(int index, double x);
        int Find(double x);
        int Delete(double x);
        void DisplayList(void);
private:
        Node* head;
};
```

Simple Linked List Class

- Operations of `List`
- `IsEmpty`: determine whether the list is empty or not
- `Insert`: insert a new node at a particular position
- `Find`: find a node with a given value
- `Delete`: delete a node with a given value
- `DisplayList`: print all the nodes in the list

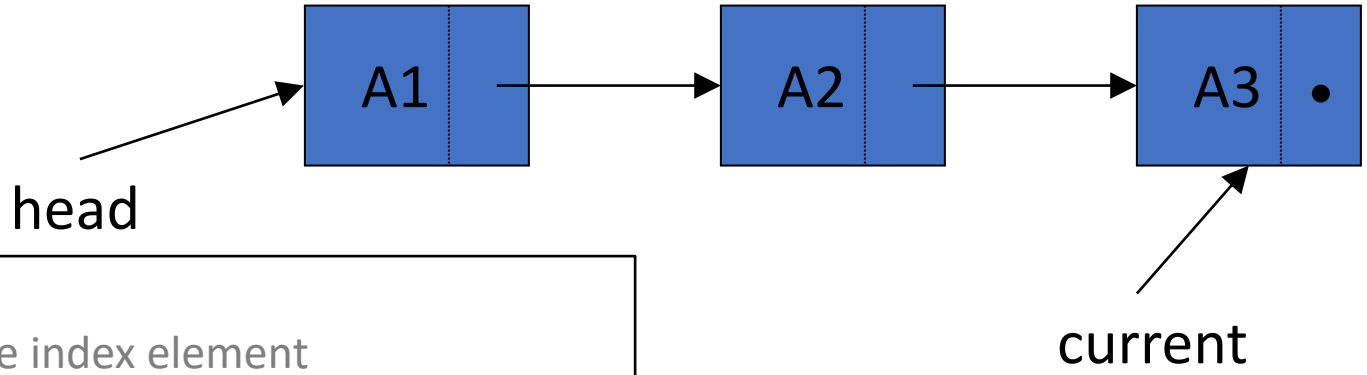
Inserting a New Node

- `bool Insert(int index, double x)`
  - Insert a node with data equal to x at the index elements
  - If the insertion is successful
    - return `true`
    - otherwise, return `false`
- If index is <= 0 or > length+1 of the list, the insertion will fail

- **Steps**
  1. Locate the node at the position one less than index **[list is indexed from 1 to n]**
  2. Allocate memory for the new node, copy data into node
  3. Point the new node to its successor (next node)
  4. Point the new node's predecessor (preceding node) to the new node

Insertion After The Last Element

- Suppose `current` points to the last element of the list
  - We can add a new last item x by doing this

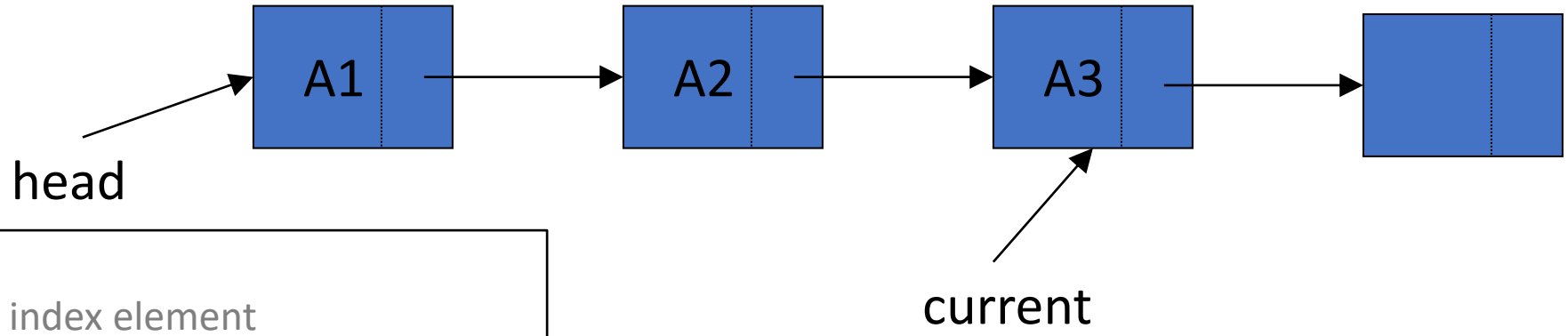


Steps
- Locate the index element
- Allocate memory for the new node
- Copy data into node
- Point the new node to its successor (next node)
- Point the new node's predecessor (preceding node) to the new node

```
current->next = new Node();
current = current->next;
current->data = x;
current->next = null;
```

Insertion After The Last Element

- Suppose `current` points to the last element of the list
  - We can add a new last item x by doing this

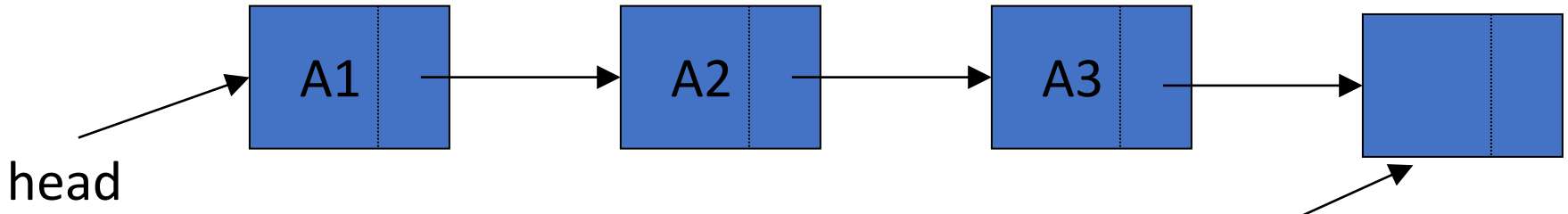


Steps
- Locate the index element
- **Allocate memory for the new node**
- Copy data into node
- Point the new node to its successor (next node)
- Point the new node's predecessor (preceding node) to the new node

```
current->next = new Node();
current = current->next;
current->data = x;
current->next = null;
```

Insertion After The Last Element

- Suppose `current` points to the last element of the list
  - We can add a new last item x by doing this

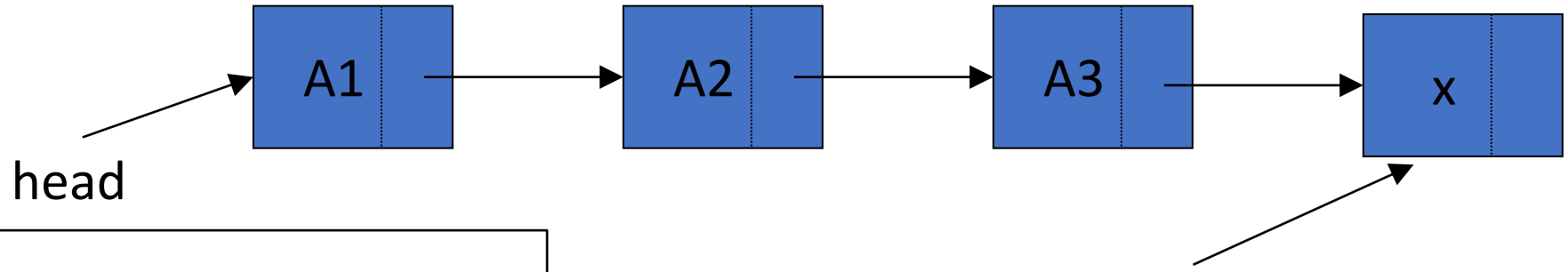


Steps
- Locate the index element
- Allocate memory for the new node
- Copy data into node
- Point the new node to its successor (next node)
- Point the new node's predecessor (preceding node) to the new node

```
current->next = new Node();
current = current->next;
current->data = x;
current->next = null;
```

Insertion After The Last Element

- Suppose `current` points to the last element of the list
  - We can add a new last item x by doing this

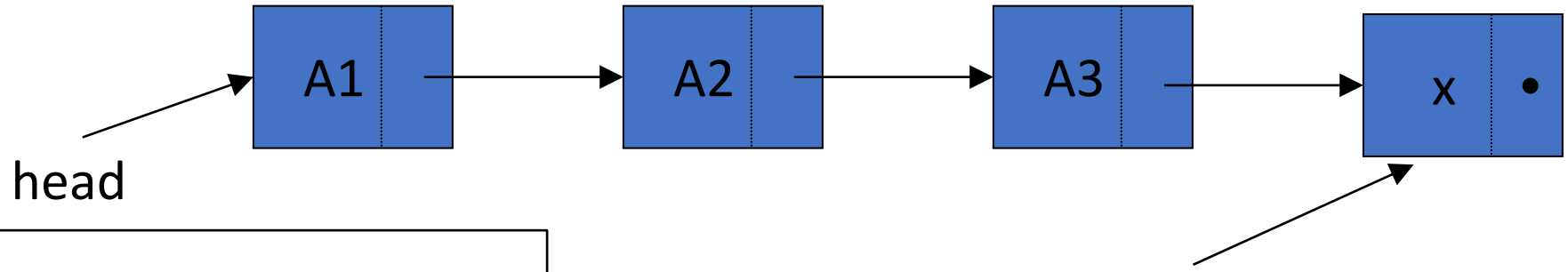


Steps
- Locate the index element
- Allocate memory for the new node
- **Copy data into node**
- Point the new node to its successor (next node)
- Point the new node's predecessor (preceding node) to the new node

```
current->next = new Node();
current = current->next;
current->data = x;
current->next = null;
```

Insertion After The Last Element

- Suppose `current` points to the last element of the list
  - We can add a new last item x by doing this

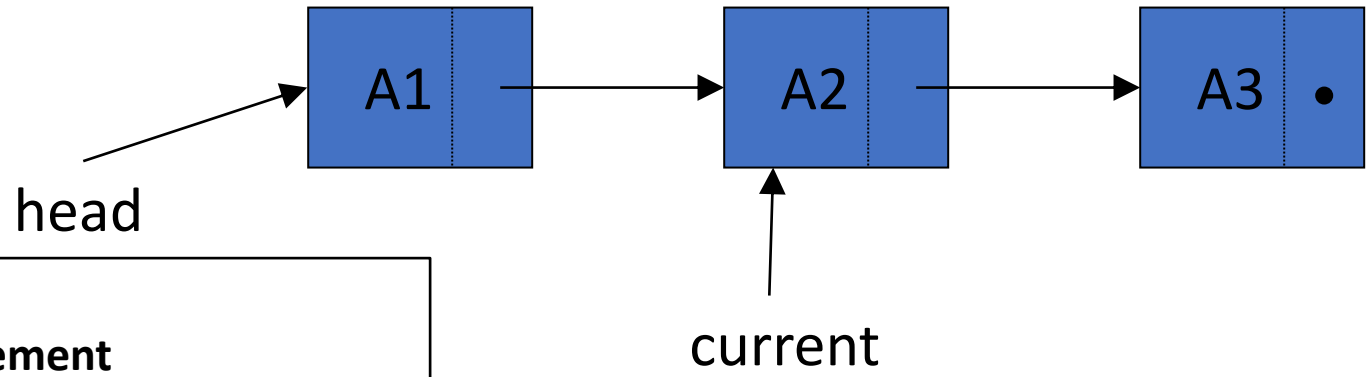


Steps
- Locate the index element
- Allocate memory for the new node
- Copy data into node
- **Point the new node to its successor (next node)**
- Point the new node's predecessor (preceding node) to the new node

```
current->next = new Node();
current = current->next;
current->data = x;
current->next = null;
```

Insert in the middle

- Suppose `current` points to the middle element of the list
  - We can add a new item x by doing this



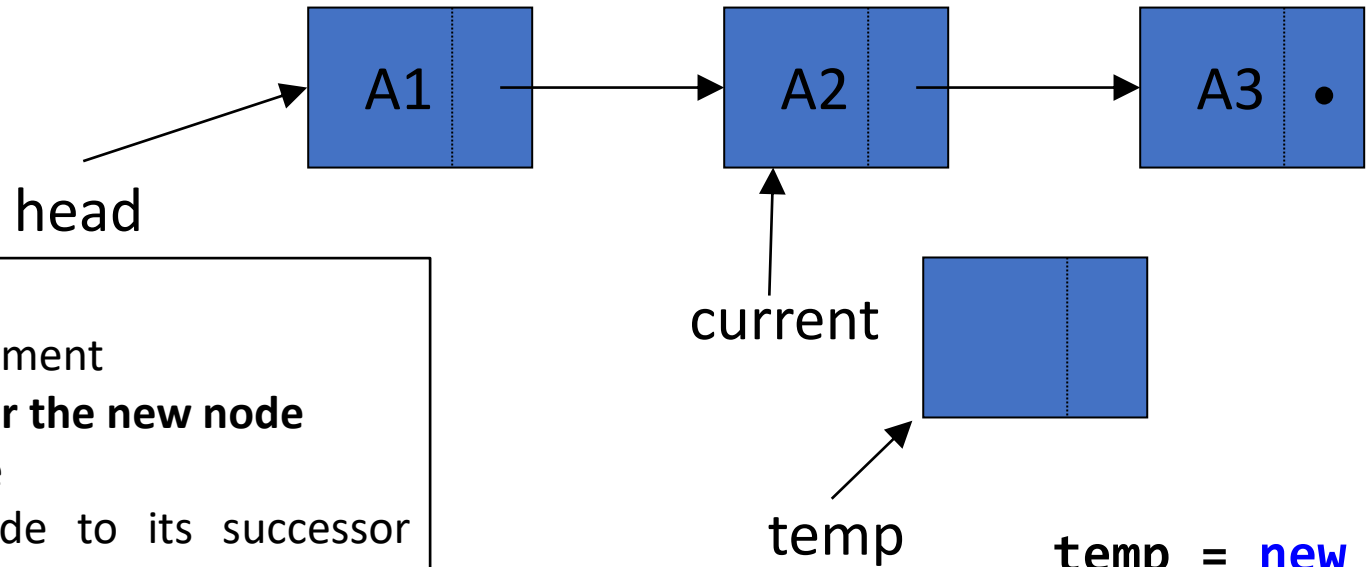
Steps

- **Locate the index element**
- Allocate memory for the new node
- Copy data into node
- Point the new node to its successor (next node)
- Point the new node's predecessor (preceding node) to the new node

```
temp = new Node();
temp -> data = x;
temp -> next = current -> next;
current -> next = temp;
```

Insert in the middle

- Suppose `current` points to the middle element of the list
  - We can add a new item x by doing this



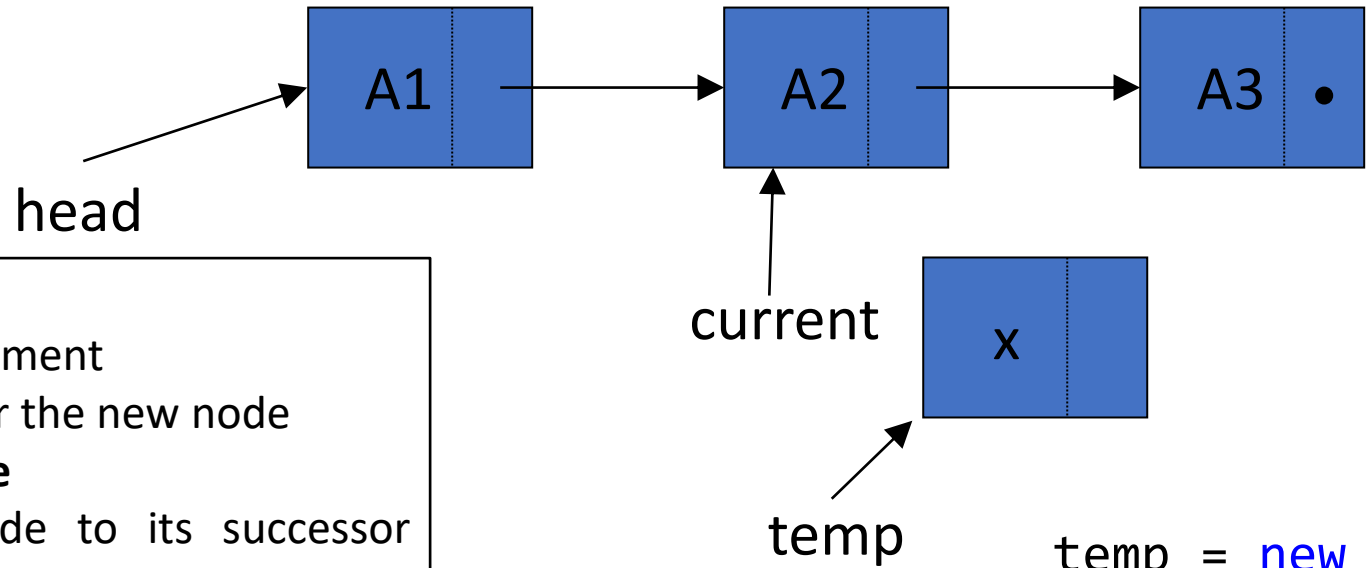
Steps

- Locate the index element
- **Allocate memory for the new node**
- Copy data into node
- Point the new node to its successor (next node)
- Point the new node's predecessor (preceding node) to the new node

```
temp = new Node();
temp -> data = x;
temp -> next = current -> next;
current -> next = temp;
```

Insert in the middle

- Suppose `current` points to the middle element of the list
  - We can add a new item x by doing this



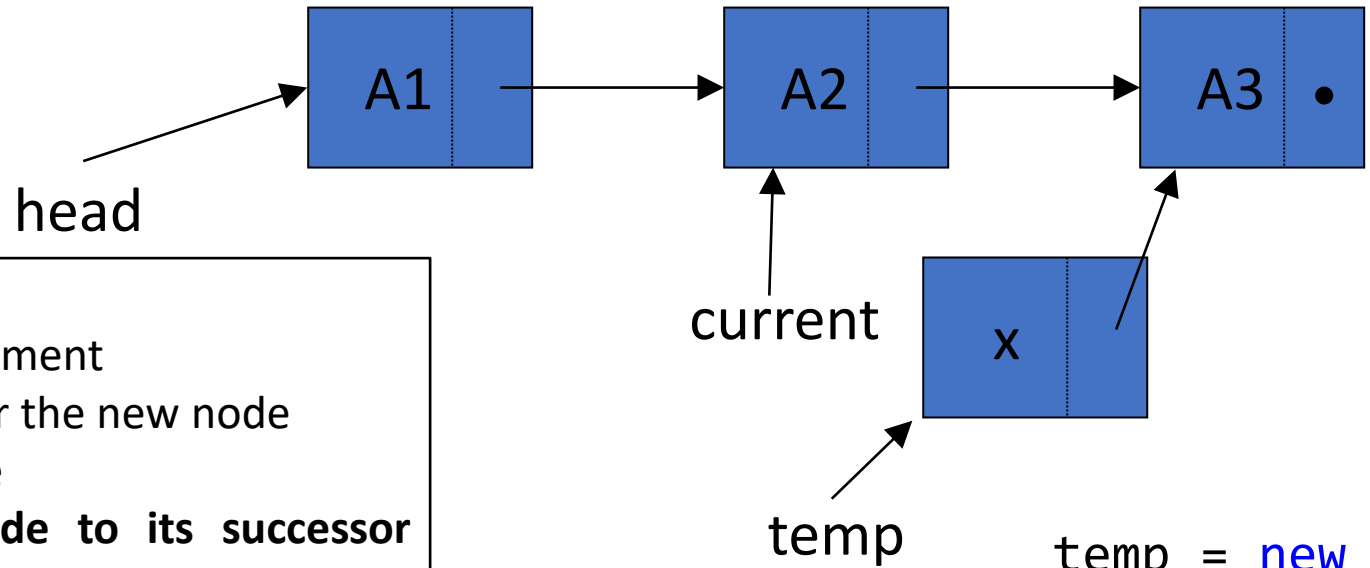
Steps

- Locate the index element
- Allocate memory for the new node
- **Copy data into node**
- Point the new node to its successor (next node)
- Point the new node's predecessor (preceding node) to the new node

```
temp = new Node();
temp -> data = x;
temp -> next = current -> next;
current -> next = temp;
```

Insert in the middle

- Suppose `current` points to the middle element of the list
  - We can add a new item x by doing this



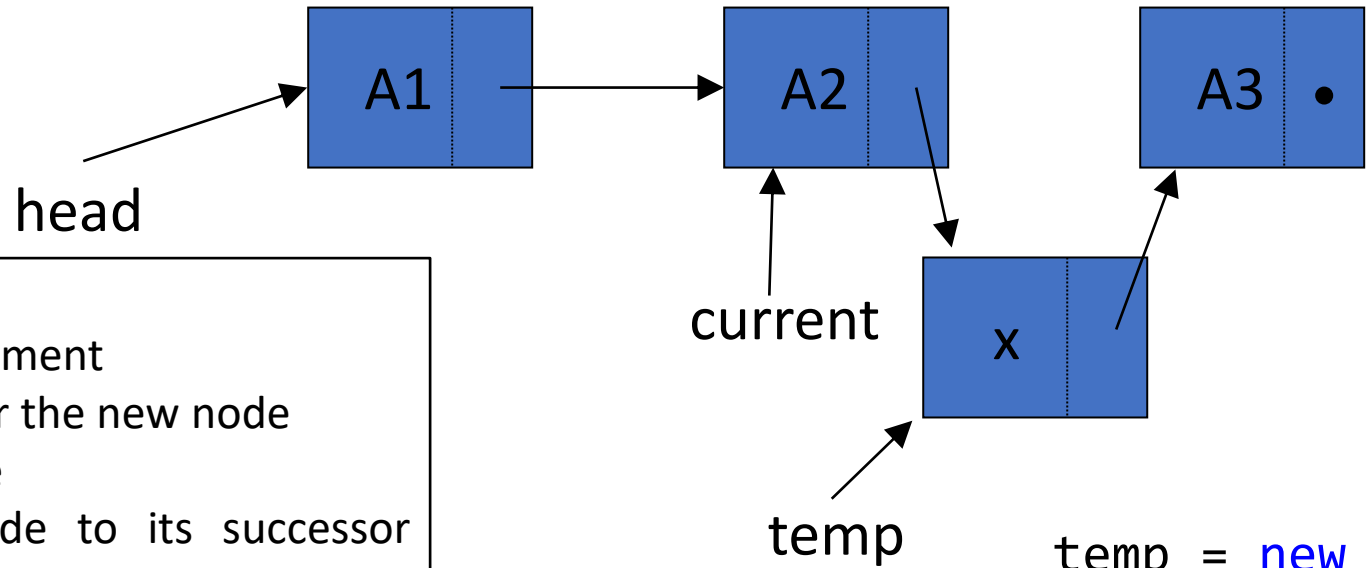
Steps

- Locate the index element
- Allocate memory for the new node
- Copy data into node
- **Point the new node to its successor (next node)**
- Point the new node's predecessor (preceding node) to the new node

```
temp = new Node();
temp -> data = x;
temp -> next = current -> next;
current -> next = temp;
```

Insert in the middle

- Suppose `current` points to the middle element of the list
  - We can add a new item x by doing this



Steps

- Locate the index element
- Allocate memory for the new node
- Copy data into node
- Point the new node to its successor (next node)
- **Point the new node's predecessor (preceding node) to the new node**

```
temp = new Node();
temp -> data = x;
temp -> next = current -> next;
current -> next = temp;
```

Inserting a New Node

- Possible cases of `Insert`
  1. Insert into an empty list
  2. Insert at front
  3. Insert at back
  4. Insert in middle

- In fact, only need to handle two cases
  - Insert as the first node (Case 1 and Case 2)
  - Insert in the middle or at the end of the list (Case 3 and Case 4)

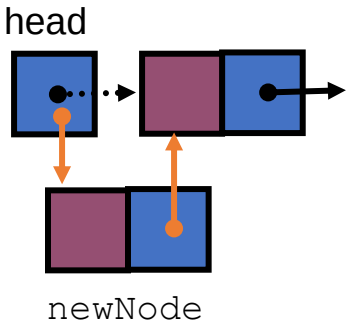
Inserting a New Node

```
bool List::Insert(int index, double x) {
          if (index <= 0) return false;

          int currIndex = 2;
          Node* current = head;
          while (current && index > currIndex) {
                    current = current->next;
                    currIndex++;
          }
          if (index > 1 && current == NULL) return false;

          Node* newNode = new Node;
          newNode->data = x;
          if (index == 1) {
                    newNode->next = head;
                    head = newNode;
          }
          else {
                    newNode->next = current->next;
                    current->next = newNode;
          }
          return true;
}
```

Insert as first element



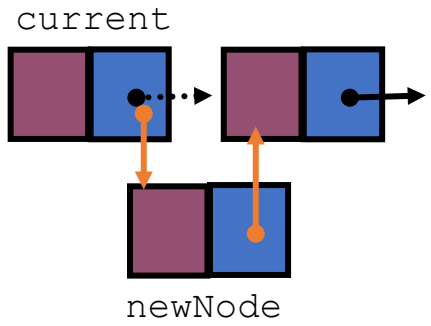
Inserting a New Node

```
bool List::Insert(int index, double x) {
        if (index <= 0) return false;

        int currIndex = 2;
        Node* current = head;
        while (current && index > currIndex) {
                current = current->next;
                currIndex++;
        }
        if (index > 1 && current == NULL) return false;

        Node* newNode = new Node;
        newNode->data = x;
        if (index == 1) {
                newNode->next = head;
                head = newNode;
        }
        else {
                newNode->next = current->next;
                current->next = newNode;
        }
        return true;
}
```

Insert after `current`



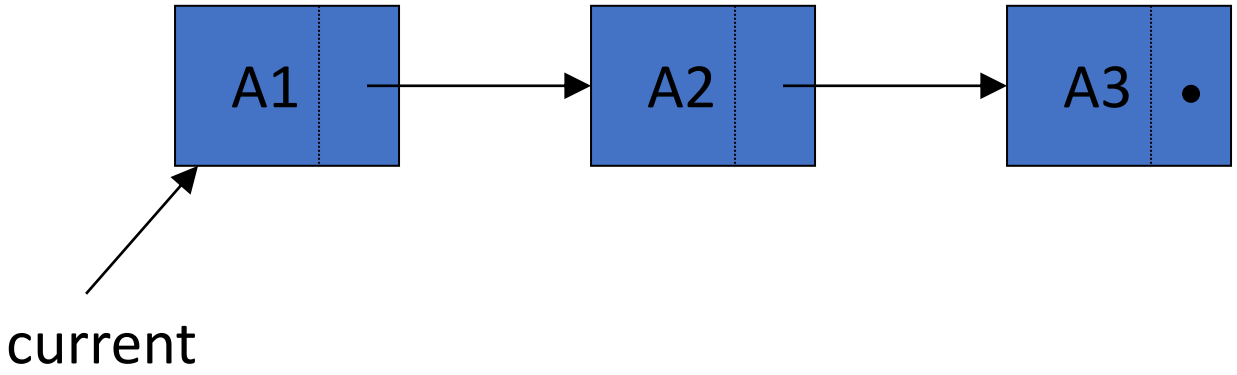
Find a node

- **`int Find(double x)`**
  - Search for a node with the value equal to x in the list
  - If such a node is found
    - Return its position
    - Otherwise, return 0

```
int List::Find(double x) {
        Node* current = head;
        int currIndex = 1;
        while (current && current->data != x) {
                current = current->next;
                currIndex++;
        }
        if (current)
                return currIndex;
        return 0;
}
```

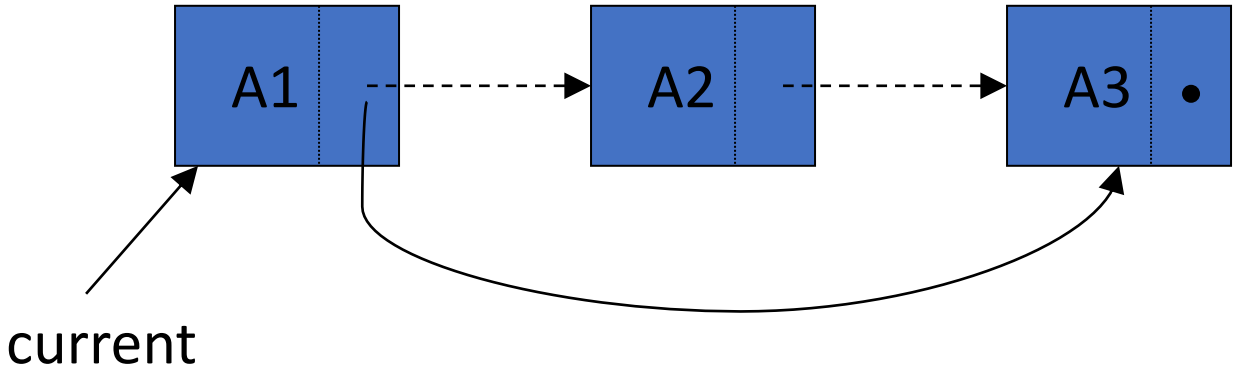
Deleting a node

- Deleting item A2 from the list



Deleting a node

- Deleting item A2 from the list

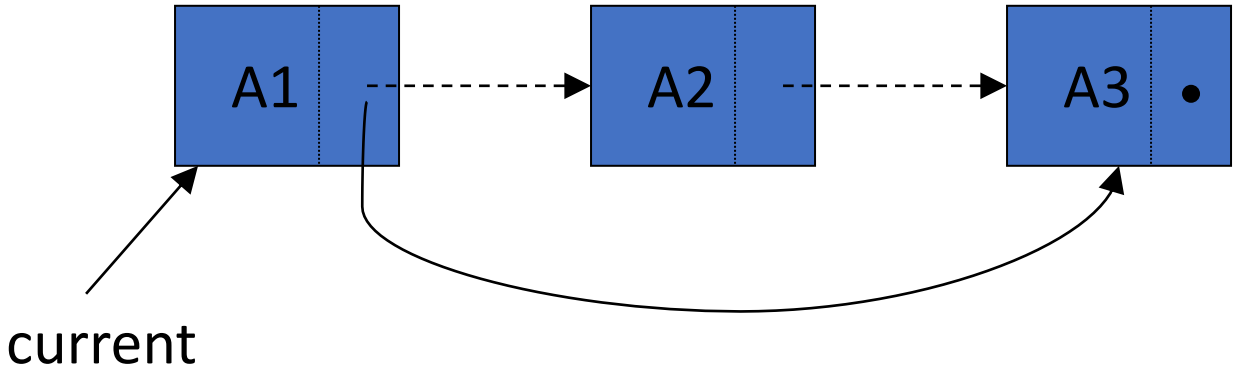



```
current->next = current->next->next;
```

Memory Leak

Deleting a node

- Deleting item A2 from the list




```
Node *deletedNode = current->next;
current->next = current->next->next;
delete deletedNode;
deletedNode = NULL;
```

No Memory Leak

Deleting a Node

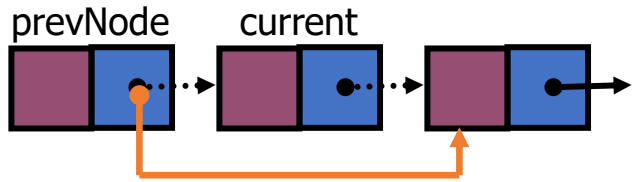
- **`int Delete(double x)`**
  - Delete a node with the value equal to x from the list
  - If such a node is found return its position
    - Otherwise, return 0

- Steps
  - Find the desirable node (similar to `Find`)
  - Set the pointer of the predecessor of the found node to the successor of the found node
  - Release the memory occupied by the found node

- Like `Insert`, there are two special cases
  - Delete first node
  - Delete the node in middle or at the end of the list

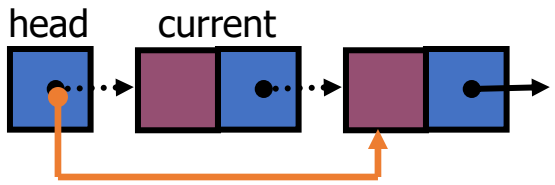
Delete a node

```
int List::Delete(double x) {
        Node* prevNode = NULL;
        Node* current = head;
        int currIndex = 1;
        while (current && current->data != x) {
                prevNode = current;
                current = current->next;
                currIndex++;
        }
        if (current) {
                if (prevNode) {
                        prevNode->next = current->next;
                        delete current;
                }
                else {
                        head = current->next;
                        delete current;
                }
                return currIndex;
        }
        return 0;
}
```



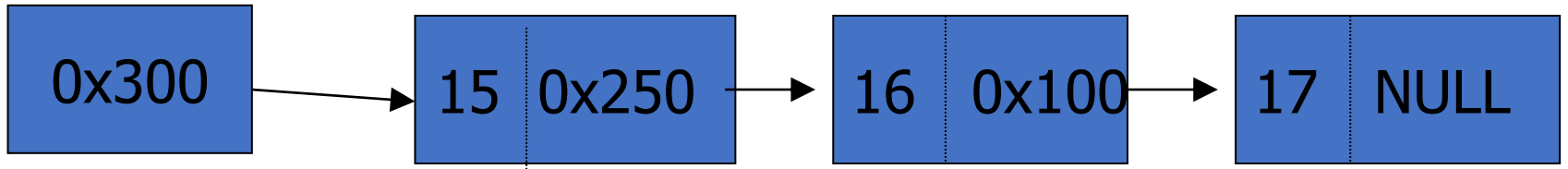
Delete a node

```
int List::Delete(double x) {
        Node* prevNode = NULL;
        Node* current = head;
        int currIndex = 1;
        while (current && current->data != x) {
                prevNode = current;
                current = current->next;
                currIndex++;
        }
        if (current) {
                if (prevNode) {
                        prevNode->next = current->next;
                        delete current;
                }
                else {
                        head = current->next;
                        delete current;
                }
                return currIndex;
        }
        return 0;
}
```



Quick Participation-2

1. Assuming that your list is empty try deleting a node with data=15.5 (i.e., call *delete(15.5)*)
2. For the following disjoint tasks assume the list be:



head 0x300 0x250 0x100

i. Try deleting node with data=15
ii. Try deleting node with data=16
iii. Try deleting node with data=17

Printing All The Elements

- **`void DisplayList(void)`**
  - Print the data of all the elements
  - Print the number of the nodes in the list

```
void List::DisplayList()
{
        int num = 0;
        Node* current = head;
        while (current != NULL) {
                cout << current->data << endl;
                current = current->next;
                num++;
        }
        cout << "Number of nodes in the list: " << num << endl;
}
```

Destroying the List

- **`~List(void)`**
  - Use the destructor to release all the memory used by the list
  - Step through the list and delete each node one by one

```
List::~List(void) {
    Node* current = head;
    Node* nextNode = NULL;
    while (current != NULL)
    {
        nextNode  =  current->next;
        delete current; // destroy the current node
        current  =  nextNode;
    }
}
```